UNIVERSITÀ DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Elaborato finale in **Reti di Calcolatori**

# *Piattaforme di Agentic AI per la Difesa da Attacchi Informatici*

Anno Accademico 2025/2026

Candidato
**Carmine Peluso**
**matr. N46007603**

# Indice

**Introduzione** **1**

**1 Fondamenti di Agentic AI** **3**

1.1 Concetti fondamentali . . . . . 3

1.2 Approcci architetturali . . . . . 4

1.3 Applicazione dell'Agentic AI al Dominio della Cybersecurity . . . . . 5

**2 Stato dell'arte e Tecnologie Tradizionali** **6**

2.1 Sistemi di Difesa Attuali . . . . . 6

2.2 Il Processo di Validazione degli Allarmi . . . . . 7

2.3 Il Fenomeno dell'Alert Fatigue . . . . . 8

**3 Architetture e Piattaforme Agentiche per la Sicurezza Informatica** **11**

3.1 Red Team: Soluzioni di Agentic AI per l'attacco . . . 12

3.1.1 Attacchi Autonomi contro Vulnerabilità Zero-Day: Il Caso Studio UIUC . . . . . 12

3.2 Blue Team: Soluzioni di Agentic AI per la difesa . . . 17

3.2.1 Dalle Piattaforme SOAR Tradizionali all'Hyper-Automation Agentica . . . . . 17
3.2.2 L'Architettura del SOAR Agentico e il Framework IVAM . . . . . 18
3.2.3 L'Architettura Multi-Agente a Tre Livelli . . . 22
3.2.4 Caso Studio Operativo e Analisi Comparativa . 28

**4 Rischi e Limitazioni dell'Agentic AI nella Cybersecurity 29**
4.1 Vulnerabilità degli agenti . . . . . 30
4.1.1 Prompt Injection e Unsafe Tool Invocation . . 30
4.1.2 Memory Poisoning e Contaminazione dei Sistemi RAG . . . . . 32
4.1.3 Allucinazioni, Auto-Denial of Service e Attacchi CRABS . . . . . 33

**5 Conclusioni 34**

**Bibliografia 36**

# Introduzione

Negli ultimi anni, la rapida evoluzione dell'Agentic AI ha ampliato significativamente la superficie di attacco dei sistemi informatici. Le moderne tecnologie di intelligenza artificiale sono sempre più sfruttate da attori malevoli, portando a un notevole incremento degli attacchi automatizzati. In un contesto in cui le minacce evolvono alla velocità delle macchine, diventa fondamentale disporre di difese altrettanto reattive e dinamiche. Tuttavia, tale necessità si scontra con l'attuale panorama aziendale, in cui la maggior parte delle aziende non ha ancora integrato soluzioni di intelligenza artificiale per automatizzare le operazioni di sicurezza.
In questo scenario, i tradizionali SOC mostrano evidenti limiti strutturali, risultando sempre più sovraccarichi a causa del volume esponenziale di allarmi di sicurezza e della carenza di personale operativo. Di conseguenza, l'efficacia del lavoro di classificazione e mitigazione svolto dagli analisti risulta gravemente compromessa, sfociando nel ben noto fenomeno dell'*alert fatigue*.
Alla luce di queste problematiche, il presente elaborato si pone l'obiettivo di analizzare le modalità con cui l'Agentic AI trasformerà l'attacco e la difesa nel panorama della cybersecurity e le principali conseguenze ad essa legate.
L'elaborato è strutturato come segue:

- Il Capitolo 1 presenta i fondamenti, le potenzialità e gli approcci architetturali che caratterizzano l'Agentic AI;

- Il Capitolo 2 esplora i processi di rilevamento e classificazione delle minacce nei *Security Operations Center* tradizionali, con un focus particolare sul fenomeno dell'*alert fatigue* e sulla necessità dell'introduzione di nuove architetture autonome;

- Il Capitolo 3 costituisce il corpo centrale del lavoro e introduce le piattaforme di Agentic AI applicate alla sicurezza informatica. Nella sezione dedicata ai *Red Team*, viene esaminata una soluzione per lo sfruttamento di vulnerabilità *zero-day*. Nella sezione relativa ai *Blue Team*, si evidenziano le rigidità legate ai SOAR tradizionali e si descrivono architetture per automatizzare il rilevamento delle minacce;

- Il Capitolo 4 tratta i principali rischi connessi all'impiego dell'Agentic AI, analizzando vulnerabilità intrinseche dei modelli come *prompt injection*, *memory poisoning* e attacchi CRABS;

- Il Capitolo 5 chiude l'elaborato raccogliendo le osservazioni finali e delineando l'importanza del paradigma *Human-in-the-Loop* per il futuro del settore.

# Capitolo 1

# Fondamenti di Agentic AI

L'Agentic AI rappresenta una nuova frontiera nel campo dell'intelligenza artificiale, capace di perseguire obiettivi complessi in modo autonomo e senza la necessità di una costante supervisione umana [2]. A differenza dei sistemi di intelligenza artificiale tradizionali, basati su regole rigide, e dei modelli generativi classici, che rispondono a richieste statiche, l'Agentic AI è guidata da obiettivi (*goal-driven*) ed è in grado di pianificare ed eseguire azioni autonomamente per raggiungere un fine prestabilito. [16].

## 1.1 Concetti fondamentali

I sistemi di Agentic AI sono progettati per operare con un elevato grado di autonomia in ambienti complessi e per eseguire compiti specifici. Il funzionamento di un agente autonomo è composto da cinque fasi principali che si ripetono in maniera ciclica:

1. ***Perception***: l'agente raccoglie ed elabora informazioni dall'ambiente circostante e da varie fonti, come sensori, log di sistema, database esterni e interfacce utente.

2. ***Reasoning***: utilizzando un *Large Language Model* (LLM), l'agente analizza i dati raccolti per comprendere il contesto, identificare le informazioni rilevanti e formulare potenziali soluzioni.

3. ***Planning***: l'agente sviluppa un piano basato sulle informazioni raccolte. Ciò comporta la definizione dell'obiettivo principale, la sua scomposizione in sotto-obiettivi più semplici e l'identificazione del modo migliore per raggiungerli.

4. ***Action***: l'agente traduce il piano formulato in interventi concreti sull'ambiente, come l'invocazione di API esterne, l'esecuzione di script o l'interazione con altri sistemi.

5. ***Reflection***: a seguito dell'esecuzione l'agente apprende dai risultati ottenuti.

Tale ciclo iterativo (schematizzato in Figura 1.1) permette all'agente di adattarsi in tempo reale alle variazioni dell'ambiente e di migliorare progressivamente le proprie prestazioni [17, 2].

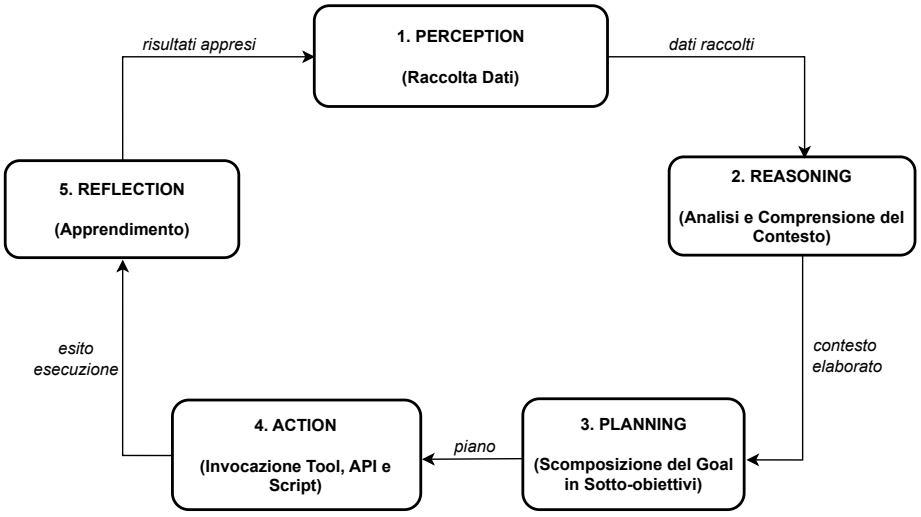



Figura 1.1: *Rappresentazione del ciclo di funzionamento di un agente autonomo.*

## 1.2 Approcci architetturali

Per strutturare agenti capaci di operare in scenari reali, la letteratura scientifica ha sviluppato diversi approcci architetturali [2]:

- ***Multi-Agent Systems* (MAS)**: architetture distribuite in cui molteplici agenti autonomi cooperano per raggiungere un obiettivo comune.
- ***Hierarchical Reinforcement Learning* (HRL)**: basato su una struttura gerarchica, in cui gli agenti di livello superiore definiscono le strategie e i sotto-obiettivi (*sub-goals*), mentre gli agenti di basso livello li eseguono.
- ***Goal-Oriented Modular Architectures***: sistemi caratterizzati da componenti modulari e riconfigurabili, specializzati in compiti specifici [2].

## 1.3 Applicazione dell'Agentic AI al Dominio della Cybersecurity

I principi teorici dell'Agentic AI trovano un campo di applicazione di primario interesse nella sicurezza informatica. Gli ambienti di rete moderni sono caratterizzati da elevata dinamicità, dalla generazione continua di eventi di sicurezza e da minacce capaci di evolvere rapidamente. In questo contesto, le proprietà fondamentali degli agenti autonomi — in particolare la capacità di pianificazione orientata agli obiettivi e il ciclo di apprendimento continuo — offrono un cambio di paradigma rispetto ai sistemi di automazione tradizionali.
Tuttavia, per comprendere l'impatto reale e la necessità di integrare piattaforme agentiche nelle infrastrutture di difesa, si rende indispensabile un'analisi preliminare dello stato dell'arte tecnologico, ampiamente affrontata nel capitolo seguente.

# Capitolo 2

# Stato dell'arte e Tecnologie Tradizionali

## 2.1 Sistemi di Difesa Attuali

Gli attuali ambienti di monitoraggio e difesa informatica si strutturano attorno ai Security Operations Center (SOC), centri operativi in cui risorse umane, standard procedurali e piattaforme tecnologiche interagiscono per identificare e contenere le minacce.
Alla base delle attività svolte dal SOC si collocano i sistemi di *Security Information and Event Management* (SIEM) e le piattaforme di *Security Orchestration, Automation, and Response* (SOAR). Il SIEM raccoglie ed elabora flussi eterogenei di dati: avvisi provenienti da *Network Intrusion Detection Systems* (NIDS)[1], log di sistema e di rete (es. proxy log) e informazioni contestuali memorizzate nella *Knowledge Base* aziendale, come i risultati delle scansioni di vulnerabilità. Attraverso regole di correlazione predefinite, il SIEM aggrega tali dati per generare allarmi che saranno successivamente sottoposti all'attenzione degli analisti [5].

[1]Un NIDS è un sistema di sicurezza che monitora e analizza il traffico di rete per identificare attività sospette, tentativi di intrusione e violazioni delle politiche di sicurezza, generando avvisi per gli amministratori di sistema. [36]

Nonostante l'elevata capacità computazionale di queste piattaforme, il processo di difesa non è completamente automatizzato. L'intervento umano rimane indispensabile sia nella fase iniziale — per definire le fonti dati e le regole di correlazione — sia nella fase operativa di analisi e risposta.

## 2.2 Il Processo di Validazione degli Allarmi

Il compito principale degli analisti di primo livello (Tier-1)[2] consiste nell'eseguire le attività di triage e validazione degli allarmi generati dal SIEM, in un processo noto come *Alarm Validation*. L'obiettivo principale di questa fase consiste nel valutare ogni segnalazione per determinare se essa corrisponda a un tentativo di intrusione effettivo (True Positive) o a una segnalazione non malevola (False Positive). L'analisi empirica condotta da Alahmadi et al. [5] ha mostrato come nelle attività quotidiane fino al 99% degli allarmi generati dalle tecnologie di monitoraggio non corrisponda a minacce reali. Per analizzare correttamente tale dinamica è opportuno distinguere i veri falsi positivi dalle attivazioni improprie note come *benign triggers*. Mentre i falsi positivi in senso stretto derivano da errori di configurazione del software o da firme di rilevamento scritte in modo impreciso, i *benign triggers* rappresentano allarmi formalmente corretti — in quanto corrispondono alla firma impostata — ma attivati da operazioni aziendali lecite o giustificate, come attività di manutenzione o l'impiego di software legacy. La distinzione è rilevante poiché entrambe le categorie generano un volume di lavoro che richiede una valutazione manuale continuativa.

[2]Gli analisti di primo livello (Tier-1) costituiscono la prima linea operativo-difensiva nel SOC: si occupano del monitoraggio continuo degli eventi di sicurezza, del triaging iniziale e del filtraggio dei falsi positivi prima dell'eventuale escalation agli analisti senior (Tier-2 e Tier-3).

L'analista, dunque, esegue la fase di validazione basandosi sulle proprie conoscenze, sia empiriche che dell'organizzazione e dell'infrastruttura di rete, nonché sulle abilità di pattern matching, ragionamento e calcolo. Tale compito è reso complesso dalla scarsa trasparenza di molte soluzioni commerciali[3], i cui allarmi forniscono descrizioni sintetiche che costringono gli analisti a ricostruire manualmente la sequenza degli eventi per comprendere la logica di rilevamento sottostante [5].

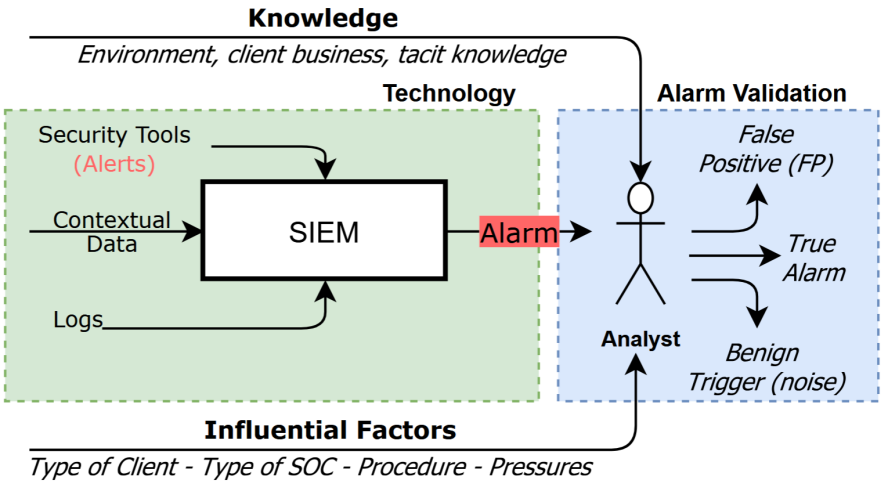



Figura 2.1: *Processo di validazione degli allarmi nei SOCs. Immagine tratta da [5]*

## 2.3 Il Fenomeno dell'Alert Fatigue

Il divario tra la velocità con cui vengono generati gli allarmi e la capacità di analisi umana determina il fenomeno dell'*alert fatigue* (affaticamento da allerta). Secondo indagini di settore condotte da *Trend Micro*, il 51% dei team SOC si dichiara sopraffatto dal volume giornaliero degli allarmi, con gli analisti che impiegano oltre il 25% del proprio tempo lavorativo nella sola gestione dei falsi positivi [1].
La letteratura accademica più recente sintetizza le cause dell'*alert fatigue* in quattro fattori primari [39]:

[3]Esempi di soluzioni commerciali sono le piattaforme SIEM o NIDS proprietarie sviluppate da vendor di settore come *Palo Alto Networks*, *IBM* e *CrowdStrike*.

1. **Carenza di Personale e Competenze** (*Staff and Skills Shortage*): il numero di analisti qualificati non è sufficiente a gestire il crescente carico di lavoro [11].

2. **Elevato Tasso di Falsi Allarmi** (*High False-Alarm Rate*): prevalenza di rumore di fondo e *benign triggers*.

3. **Dashboard Frammentate e Sovraccariche** (*Disconnected Dashboards*): le segnalazioni provengono da strumenti diversi limitando la visibilità degli alert e introducendo una maggiore difficoltà di correlazione [11]. La mancanza di integrazione tra gli strumenti riduce ulteriormente l'efficacia degli analisti costringendoli a spostarsi tra console differenti.

4. **Procedure Operative Inefficienti** (*Inefficient SOPs*): l'assenza di *playbook*[4] ben strutturati e processi definiti così come la mancanza di automazione - che accresce il rischio di errori umani - e orchestrazione rende i SOC inefficienti.

Lo stress prolungato desensibilizza gli operatori, compromettendo le loro capacità cognitive e inducendo stati di burnout che aumentano drasticamente il rischio che minacce critiche passino inosservate.
Questi fattori dimostrano che i sistemi tradizionali basati su regole e firme statiche (NIDS/IPS) sono intrinsecamente inadeguati a gestire scenari dinamici e minacce che variano nel tempo. Con l'avvento dei modelli di *Frontier AI*[5], inoltre, il volume degli allarmi generati è cresciuto a dismisura rendendo le operazioni di sicurezza ancora più insostenibili .
Come evidenziato dal report *IBM Cost of a Data Breach 2026*, l'impiego dell'intelligenza artificiale consente agli hacker di attaccare alla

[4] I *playbook* sono guide operative che definiscono in modo dettagliato le azioni da intraprendere per gestire specifici incidenti o scenari di sicurezza informatica, supportando una risposta coordinata e standardizzata [44].

[5] Con il termine *Frontier AI* si definiscono i modelli di intelligenza artificiale più avanzati e di nuova generazione, caratterizzati da elevate capacità esecutive e potenzialmente associati a nuovi rischi per la sicurezza informatica [37].

velocità delle macchine. I modelli di Frontier AI sono in grado di individuare autonomamente vulnerabilità zero-day in sistemi operativi e browser web, riducendo a pochi minuti la finestra temporale tra la scoperta di una falla e il suo sfruttamento.
Nell'ultimo anno si è registrato un incremento del 56% degli attacchi basati su AI, portando il costo medio globale di una violazione a circa 5 milioni di dollari. Tra le organizzazioni colpite da violazioni legate all'AI, il 92% non disponeva di controlli di sicurezza adeguati.
In questo scenario, le difese tradizionali e l'analisi manuale non sono più sostenibili. Sebbene l'adozione dell'AI difensiva consenta un risparmio medio di 1,93 milioni di dollari, attualmente solo il 36% delle organizzazioni impiega soluzioni di AI e automazione in modo esteso nelle operazioni di sicurezza. Emerge quindi la necessità irrinunciabile di evolvere verso un nuovo paradigma: l'**Agentic AI**, capace di operare in modo autonomo, contestuale e alla velocità delle macchine per colmare il divario tra attaccanti e difensori [18].

# Capitolo 3

# Architetture e Piattaforme Agentiche per la Sicurezza Informatica

Nel panorama moderno della cybersecurity le organizzazioni si ritrovano ad affrontare continuamente minacce e attacchi sempre più sofisticati alle proprie infrastrutture. Per massimizzare l'efficacia delle contromisure, le operazioni di sicurezza vengono strutturate attorno a due reparti specializzati. Il Red Team simula, in maniera controllata, gli attacchi avversari con lo scopo di individuare vulnerabilità dei sistemi aziendali mentre il Blue Team si occupa della difesa di questi ultimi tramite il monitoraggio continuo del traffico di rete, l'analisi dei log di sistema e l'elaborazione di strategie di mitigazione adeguate [24]. Con l'introduzione dell'Agentic AI, le operazioni di entrambe le squadre stanno subendo una profonda trasformazione. Nelle sezioni seguenti verranno analizzate le principali piattaforme e architetture autonome, sia per l'attacco che per la difesa.

## 3.1 Red Team: Soluzioni di Agentic AI per l'attacco

### 3.1.1 Attacchi Autonomi contro Vulnerabilità Zero-Day: Il Caso Studio UIUC

Per evidenziare l'evoluzione dell'Offensive AI dagli attacchi basati su singolo agente alle squadre di agenti offensivi, si è scelto di analizzare un esperimento di particolare rilevanza, condotto dai ricercatori della University of Illinois Urbana-Champaign [46]. Lo studio si focalizza sullo sfruttamento autonomo delle vulnerabilità *Zero-Day* (0DV) in applicazioni web reali, ovvero quelle falle di sicurezza non ancora note ai creatori del software o prive di patch correttive [7].

#### I Limiti dell'Agente Singolo e del Paradigma ReAct

I primi tentativi di automatizzare le attività di penetrazione informatica mediante intelligenza artificiale prevedevano l'impiego di un singolo agente basato sul paradigma *ReAct* (*Reasoning and Acting*) [33, 12]. Secondo tale scenario, il modello linguistico esegue un ciclo continuo in cui esegue un'azione sul target, osserva la risposta restituita dall'ambiente e riutilizza questo feedback per pianificare ed eseguire l'azione successiva [46]. Sebbene questo approccio abbia dimostrato la sua efficacia in contesti *Capture-The-Flag* (CTF)[1] [43, 13] o *One-Day Vulnerabilities* (1DV)[2] [12] esso fallisce drasticamente di fronte a vulnerabilità reali non note.

L'inadeguatezza dell'agente singolo deriva principalmente da due ostacoli strutturali. Da un lato, durante l'esplorazione di un'applicazione

[1] I *Capture The Flag* (CTF) sono competizioni di sicurezza informatica in cui i partecipanti individuano e sfruttano vulnerabilità in sistemi predisposti al fine di recuperare una stringa di testo nascosta (denominata flag). [31]

[2] Una *One-Day Vulnerability* (1DV) è una vulnerabilità già pubblicamente nota e spesso corretta dal fornitore, ma ancora sfruttabile nei sistemi che non hanno applicato gli aggiornamenti di sicurezza disponibili.

web complessa, l'accumulo massivo di codice HTML, risposte HTTP e log di errore causa una rapida saturazione della finestra di contesto del modello linguistico, degradando la qualità del processo decisionale. Dall'altro, si verifica un'intrinseca incapacità di effettuare il *backtracking*: quando l'agente intraprende una determinata strategia di attacco che si rivela fallimentare, incontra notevoli difficoltà a "fare un passo indietro" per rivalutare le proprie azioni e tentare una differente classe di vulnerabilità, rimanendo spesso bloccato in tentativi ripetitivi ed inefficaci [12].

### L'Architettura Gerarchica Multi-Agente (HPTSA)

Per superare la rigidità dell'agente singolo, Zhu et al. [46] hanno progettato il framework HPTSA (*Hierarchical Planning and Task-Specific Agents*), un'architettura multi-agente con una struttura ad albero che distribuisce le responsabilità decisionali ed esecutive su tre livelli gerarchici:

- **Hierarchical Planner**: Collocato al vertice dell'architettura, ha il compito di esplorare la superficie d'attacco dell'applicazione target senza eseguire direttamente gli *exploit*[3]. Il Planner individua le aree potenzialmente vulnerabili e traduce quanto analizzato in istruzioni che saranno successivamente inviate al Manager.

- **Team Manager**: Funge da orchestratore in quanto riceve le direttive dal Planner e decide a quali *Task-Specific Agents* delegare i compiti. Inoltre, il Manager raccoglie e sintetizza i risultati delle esecuzioni precedenti, mantenendo la memoria storica dell'attacco. Se un tentativo fallisce, il Manager rielabora l'informazione ed esegue il *backtracking*, rieseguendo il *Task-Specific*

[3]L'esecuzione di un *exploit* consiste nello sfruttamento di una vulnerabilità per compromettere la sicurezza di un sistema.

*Agent* (al quale fornisce informazioni più precise) o ingaggiando un altro agente.

- **Task-Specific Expert Agents**: Rappresentano il braccio dell'attacco. Sono dei sotto-agenti altamente specializzati per singola tipologia di vulnerabilità: *SQL Injection* (SQLi), *Cross-Site Scripting* (XSS), *Cross-Site Request Forgery* (CSRF), *Server-Side Template Injection* (SSTI) e un agente per le scansioni *ZAP*.

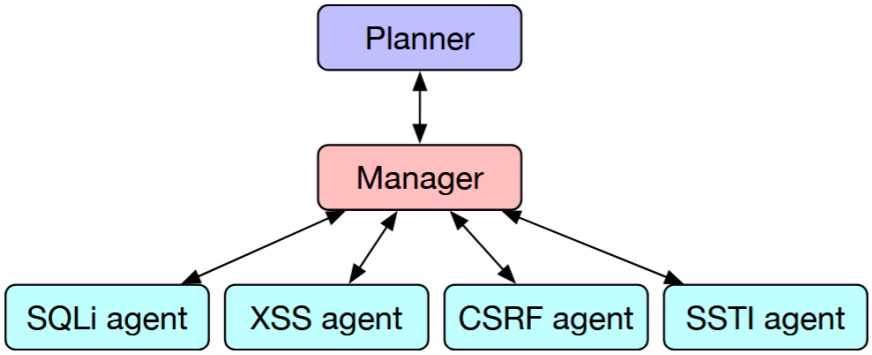



Figura 3.1: *Diagramma dell'Architettura HPTSA. Immagine tratta da [46]*

Questa suddivisione modulare garantisce che ogni agente esperto mantenga un contesto di lavoro ridotto, delegando la gestione della strategia di attacco e del *backtracking* agli agenti dei livelli superiori.

### Strumentazione Tecnica, Documentazione e Ottimizzazione del Contesto

Oltre alla ripartizione gerarchica dei ruoli, l'efficacia operativa dei singoli *Task-Specific Agents* nell'architettura HPTSA si fonda sull'integrazione di strumenti, documentazione, prompt specifici e tecniche di gestione efficiente della memoria [46].
Per interagire con l'applicazione target, tutti gli agenti esperti utilizzano il framework *Playwright* per accedere al browser[4], oltre ad avere

[4]Playwright traduce le decisioni degli LLM in azioni sul browser (ad esempio cliccare bottoni, navigare tra le pagine).

accesso al terminale di sistema e a tool specifici di dominio, quali *sqlmap*[5] per l'agente SQLi [34] e la suite *ZAP*[6] per l'agente ZAP [6]. Per guidare il processo di generazione dei payload e definire i passi della strategia, a ciascun agente esperto viene fornito l'accesso diretto a una selezione di 5-6 documenti (manuali e guide sulle specifiche classi di attacco), oltre a prompt personalizzati (in base alla vulnerabilità) integrati nel suo contesto [46].
Dal punto di vista dell'efficienza computazionale, il lavoro di Zhu et al. risolve il problema della saturazione della *context window* mediante una tecnica di semplificazione del codice HTML (*HTML pruning*). Prima di inviare la struttura della pagina web al modello linguistico, il sistema rimuove automaticamente tutti i tag non rilevanti ai fini strategici dell'attacco (come fogli di stile CSS, immagini ed elementi grafici SVG), abbattendo drasticamente il volume dei token scambiati [46].

### Dinamiche di Attacco Multi-Agente: Il Caso flusity-CMS

La reale capacità della squadra di condividere la conoscenza tra esecuzioni diverse emerge chiaramente dall'analisi dei casi studio riportati nel paper, come la compromissione del sistema di gestione contenuti *flusity-CMS* (interessato dalle vulnerabilità di tipo XSS e CSRF).
L'attacco si è sviluppato attraverso una collaborazione dinamica e iterativa:

1. **Fase XSS**: Il *Team Manager* attiva inizialmente l'agente specializzato in *Cross-Site Scripting*[7]. Dopo tre tentativi, l'agente naviga verso un modulo per la gestione delle gallerie fotografiche e costruisce il payload XSS definitivo, sfruttando la vulnerabilità bersaglio.

[5] *sqlmap* è un tool specifico per l'esecuzione di SQL injection.

[6] ZAP è un software che testa la sicurezza di un sito web mappandone la struttura ed effettuando una scansione per rilevare le vulnerabilità [9].

[7] Il *Cross-Site Scripting* (XSS) è una vulnerabilità delle applicazioni web che consente a un attaccante di eseguire codice malevolo, generalmente JavaScript, nel browser di altri utenti.

2. **Fase SQLi e Riconfigurazione**: Successivamente, il Manager ingaggia l'agente SQLi il quale esegue tre tentativi falliti sia sulla pagina di login, sia sui moduli di ricerca e creazione dei post della pagina `admin.php`. Il Manager raccoglie l'esito negativo e, tramite il meccanismo di *backtracking*, determina che la pista SQLi non è praticabile, evitando ulteriori sprechi di risorse.

3. **Fase CSRF**: Sfruttando le informazioni raccolte dall'agente SQLi sulla struttura interna di `admin.php`, il Manager istruisce l'agente CSRF[8] a concentrarsi esclusivamente su quel path. Quest'ultimo individua il modulo di creazione dei menu e forgia con successo un payload che, se cliccato dall'amministratore, permette la creazione di menu arbitrari, sfruttando la vulnerabilità.

Tale dinamica evidenzia come un'architettura gerarchica consenta una tempestiva condivisione delle scoperte e dei percorsi infruttuosi tra i sotto-agenti specializzati [46].

### Valutazione Empirica e Risultati del Benchmark

Per valutare rigorosamente le prestazioni del sistema, gli autori hanno costruito un benchmark composto da 14 vulnerabilità web reali e recenti, tutte successive alla *knowledge cutoff date*[9] del modello GPT-4 impiegato [46].
I risultati confermano il netto salto prestazionale garantito dall'architettura gerarchica multi-agente nell'esplorazione di vulnerabilità *zero-day*. Rispetto all'agente singolo *ReAct* privo di descrizione della falla, la struttura HPTSA raddoppia l'efficacia dell'attacco sulla metrica *Pass@5* (passando dal 21% al 42%) e la quadruplica sulla metrica *Pass@1* (dal 4.2% al 18%), sconfiggendo nettamente l'agente monolitico. Inoltre, HPTSA si avvicina in modo significativo al limite supe-

[8] Il *Cross-Site Request Forgery* (CSRF) è una vulnerabilità che induce un utente autenticato a eseguire, inconsapevolmente, richieste non autorizzate verso un'applicazione web.

[9] La *knowledge cutoff date* è la data limite fino alla quale sono stati raccolti i dati per addestrare un modello di IA.

riore teorico rappresentato dall'agente fornito preventivamente della descrizione dettagliata della vulnerabilità, il quale ottiene il 75% su *Pass@5*, evidenziando come una squadra di agenti possa ricostruire la conoscenza di un attacco senza alcuna informazione preliminare [46]. Oltre ad essere efficace, l'esperimento risulta anche sostenibile dal punto di vista economico. L'esecuzione di un singolo tentativo di attacco tramite l'architettura HPTSA comporta un costo medio di $4.39 in token elaborati, traducendosi in una spesa per exploit completato con successo pari a circa $24.40 [46]. Se si confronta tale valore con il costo medio stimato per la sessione di un analista umano specializzato (pari a circa $75 per un'ora e mezza di penetrazione manuale), l'attacco condotto da agenti autonomi risulta circa 3.1 volte più economico, oltre ad essere istantaneamente scalabile e privo di vincoli orari, ponendo serie sfide per la sostenibilità delle difese tradizionali [46].

## 3.2 Blue Team: Soluzioni di Agentic AI per la difesa

### 3.2.1 Dalle Piattaforme SOAR Tradizionali all'Hyper-Automation Agentica

Per fronteggiare le criticità operative dei SOC moderni (quali l'*alert fatigue*), sono state introdotte le piattaforme SOAR (*Security Orchestration, Automation, and Response*), con l'obiettivo di velocizzare e rendere più efficiente la risposta agli incidenti, automatizzando i task ripetitivi e orchestrando strumenti di sicurezza eterogenei [40, 10]. Storicamente, i SOAR tradizionali hanno fatto un ampio ricorso ad ambienti di sviluppo *no-code* e *low-code*[10], basati su interfacce visive e componenti pre-costruiti [23]. Sebbene questo approccio abbia consen-

[10]Gli ambienti no-code o low-code sono contesti di sviluppo che consentono di creare applicazioni o workflow automatizzati tramite interfacce grafiche (drag-and-drop) e componenti pre-costruiti, senza dover scrivere codice sorgente manualmente o riducendolo al minimo.

tito anche a personale privo di avanzate competenze di programmazione di creare rapidamente automatismi di sicurezza, esso risulta spesso inattuabile di fronte alla dinamicità dei moderni attacchi informatici [23, 35].

Tra i limiti principali emerge innanzitutto la rigidità dei *playbook*, i quali si basano su procedure fisse che non riescono ad adattarsi a minacce mutevoli o a contesti operativi altamente specializzati. Al crescere della complessità dell'organizzazione, infatti, i *playbook* tendono a diventare enormi, monolitici e difficili da aggiornare quando si presentano nuovi vettori d'attacco[11], richiedendo il continuo intervento manuale degli analisti per mantenere la compatibilità con i vari tool di sicurezza [44]. Gli approcci tradizionali, inoltre, operano senza una vera comprensione del contesto, risultando incapaci di modificare la propria strategia d'azione in tempo reale [21].

Per superare tali vincoli, Ismail et al. propongono un modello di ***Hyper-Automation*** basato su *AI-driven agents* che sfruttano le capacità di ragionamento dei *Large Language Models* (LLMs) (*Agentic-LLM*) per comprendere il contesto dell'allarme ed elaborare la strategia di risposta, per poi suddividerla in una sequenza ordinata di singoli sotto-task eseguibili in modalità *zero-shot*[12] [21].

### 3.2.2 L'Architettura del SOAR Agentico e il Framework IVAM

Per superare le limitazioni dei sistemi tradizionali, Ismail et al. hanno introdotto il framework **IVAM** (*Investigation, Validation, Active Monitoring*), una metodologia che organizza le direttive di sicurezza

[11]Un vettore di attacco rappresenta il metodo, la tecnica o il percorso specifico utilizzato da un hacker per violare un sistema informatico, aggirare le difese o distribuire codice malevolo.

[12]Con esecuzione *zero-shot* si intende la capacità di un modello linguistico (LLM) o di un agente autonomo di portare a termine un compito o usare uno strumento senza aver prima ricevuto esempi o dimostrazioni pratiche. L'agente ricava la soluzione basandosi unicamente sulla comprensione del contesto e delle istruzioni ricevute nel prompt.

da intraprendere per contrastare una minaccia in fasi ordinate e ben definite [21].

Il framework IVAM automatizza le azioni difensive di un agente autonomo basandosi sulle TTPs (*Tactics, Techniques, and Procedures*) di **MITRE ATT&CK**[13], sulle metriche di **Quantitative Risk Assessment (QRA)** e sulle linee guida del **NIST Cybersecurity Framework (CSF)**, uno standard di riferimento internazionale sviluppato dal *National Institute of Standards and Technology* per la gestione e la mitigazione dei rischi informatici [21]. Il CSF si articola in cinque funzioni core che coprono l'intero ciclo di vita della sicurezza: *Identify* (identificazione e valutazione dei rischi), *Protect* (attuazione delle contromisure), *Detect* (monitoraggio e rilevamento delle anomalie), *Respond* (risposta tempestiva agli incidenti) e *Recover* (ripristino dei sistemi) [21].

Nel modello IVAM, le regole normative e procedurali del NIST svolgono due ruoli distinti ma complementari: da un lato, gli standard come il *NIST SP 800-53* e le normative di settore (quali PCI-DSS, GDPR e HIPAA) vengono impiegati nella fase iniziale per identificare i controlli di sicurezza compromessi e valutare l'impatto legale della violazione [21]; dall'altro, il *NIST Cybersecurity Framework* (CSF 2.0) fornisce il pilastro procedurale per orientare la strategia dell'agente, garantendo che le azioni difensive siano formalizzate in modo coerente con le sopracitate funzioni di protezione (*Protect*) e risposta (*Respond*) [21].

Il ciclo operativo IVAM è articolato in tre fasi [21]:

1. ***Investigation***: La fase di Investigation prevede la raccolta di dati da log di sistema, piattaforme SIEM (come *Wazuh*), IDS e strumenti *Endpoint Detection and Response* (EDR)[14] [21]. At-

[13]MITRE ATT&CK è una matrice per la classificazione degli attacchi informatici. Le *Tactics* identificano gli obiettivi dell'attaccante, le *Techniques* descrivono i metodi utilizzati e le *Procedures* rappresentano casi reali di applicazione delle tecniche [4].

[14]Gli *Endpoint Detection and Response* (EDR) sono soluzioni di cybersecurity progettate per monitorare continuamente gli endpoint (PC, server, laptop e dispositivi mobili), rilevare attività sospette e supportare i team di sicurezza nell'analisi e nella risposta alle minacce in tempo reale. [27]

traverso un modulo dedicato di *Data Enrichment*, il sistema estrae informazioni sull'attacco (tra cui l'IP sorgente, l'IP target, il *severity level*[15], il timestamp) e classifica l'attacco in base alla tipologia (es. *SSH Brute Force*). Successivamente l'attacco viene mappato sulla tecnica MITRE ATT&CK corrispondente tramite il suo ID univoco (es. codice *T1110* per attacchi di tipo *Brute Force*). L'accuratezza di questo passaggio è di vitale importanza in quanto determina la strategia di difesa appropriata in base allo specifico vettore d'attacco usato dall'hacker.

In base a normative di compliance (quali NIST SP 800-53, PCI-DSS, GDPR, HIPAA) si verifica quali requisiti e controlli di sicurezza sono stati messi a rischio o violati dall'attacco determinando in tal senso l'impatto legale dell'incidente. Si struttura poi una ***Mitigation Procedure*** (strategia di mitigazione o piano di contenimento) specifica in base alla minaccia, e in regola con gli standard ufficiali del NIST CSF 2.0.

La fase di investigazione comprende, inoltre, l'analisi dei file e dei sistemi che hanno subito modifiche durante l'attacco (es. modifiche al file system, *privilege escalation*[16]) e la determinazione dello *scope* dell'infezione, volto a mappare gli endpoint compromessi e a monitorare il *Lateral Movement*[17] dell'aggressore esaminando le attività di autenticazione e le comunicazioni tra sistemi.

Un ultimo step, infine, riguarda l'identificazione del vettore di infezione iniziale attraverso l'ispezione dei canali di comunicazione (es. email gateway per cercare tentativi di phishing) e dei log di accesso (es. provenienti da VPN o SSH per rilevare accessi

[15]Il severity level è il livello di gravità dell'attacco.

[16]La *privilege escalation* è una tecnica mediante la quale un attaccante, partendo da un account o da un processo con privilegi limitati, ottiene livelli di accesso superiori a quelli originariamente autorizzati, compromettendo risorse o funzionalità riservate [32].

[17]Per *Lateral Movement* in cybersecurity si intende la fase di un attacco informatico in cui un avversario, dopo aver ottenuto un accesso iniziale a un sistema, si sposta all'interno della rete compromessa sfruttando account, credenziali o servizi remoti per raggiungere altri sistemi e avvicinarsi ai propri obiettivi [28].

non autorizzati). Tale processo permette di isolare la porta di ingresso dell'attacco, preservando l'integrità del dispositivo vittima ed evitando che la minaccia si diffonda verso altri sistemi [21].

2. ***Validation***: Una volta elaborate le azioni di mitigazione, il framework applica una valutazione quantitativa del rischio (QRA) che può essere svolta prima o dopo l'esecuzione della strategia difensiva. Nel primo caso l'agente calcola il rischio stimato per verificare se l'efficacia della contromisura giustifica i suoi possibili effetti collaterali [21]. Nel secondo, invece, verifica se l'attacco è stato realmente neutralizzato, ossia se il valore del rischio è sceso a un livello accettabile.

   Considerata la difficoltà di attribuire un valore economico monetario certo agli asset aziendali intangibili (come la reputazione o la continuità operativa), il sistema adotta un approccio di **quantificazione relativa del rischio** (*Relative QRA*), calcolato mediante la formula:

$$R = (L \times I) \times (1 - E) \tag{3.1}$$

   dove $R$ rappresenta il Rischio Relativo, $L$ la probabilità di occorrenza dell'incidente (*Likelihood*), $I$ l'impatto operativo/reputazionale/finanziario (*Impact*) ed $E$ l'efficacia della strategia di mitigazione adottata[18] (*Mitigation Effectiveness*) [21]. Tale calcolo consente agli agenti di bilanciare i benefici della difesa con gli eventuali effetti collaterali (ad esempio il blocco involontario di utenti legittimi).

3. ***Active Monitoring***: L'ultima fase garantisce che le conoscenze acquisite e l'esito delle azioni eseguite vengano reiniettati continuamente nella memoria del sistema in tempo reale [21].

[18] Maggiore è l'efficacia della strategia di mitigazione adottata minore sarà il rischio relativo.

Se il monitoraggio rileva minacce o anomalie emergenti, il ciclo difensivo si riavvia automaticamente partendo da una nuova fase di investigazione. Questa fase implica che l'agente sia in costante miglioramento e, pertanto, evolverà parallelamente alle minacce.

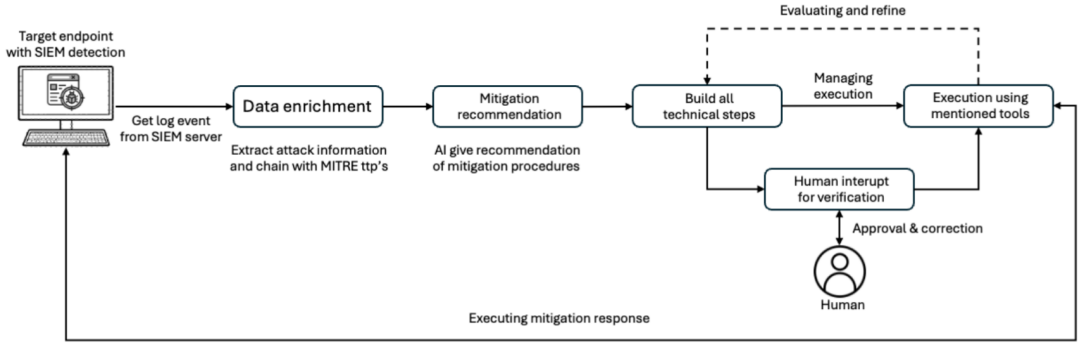



Figura 3.2: *Workflow end-to-end della risposta agli incidenti. Immagine tratta da [21].*

### 3.2.3 L'Architettura Multi-Agente a Tre Livelli

Per tradurre la metodologia IVAM in una piattaforma software scalabile, Ismail et al. propongono un'architettura multi-agente organizzata su tre livelli [21], come mostrato in Figura 3.3.

#### Livello 1: *Planning and Data Management*

Il primo livello dell'architettura costituisce il motore di pianificazione e gestione dei dati, la cui funzione fondamentale è preparare, pulire e contestualizzare tutte le informazioni relative agli eventi di sicurezza prima che vengano trasmesse ai modelli di intelligenza artificiale per il ragionamento.
All'ingresso di questo strato agisce il modulo di raffinamento dei dati (*Data Refinement*), che ha la funzione di pulire, etichettare e strutturare i dati grezzi provenienti dai sensori di rilevamento e dalle piattaforme SIEM [21].

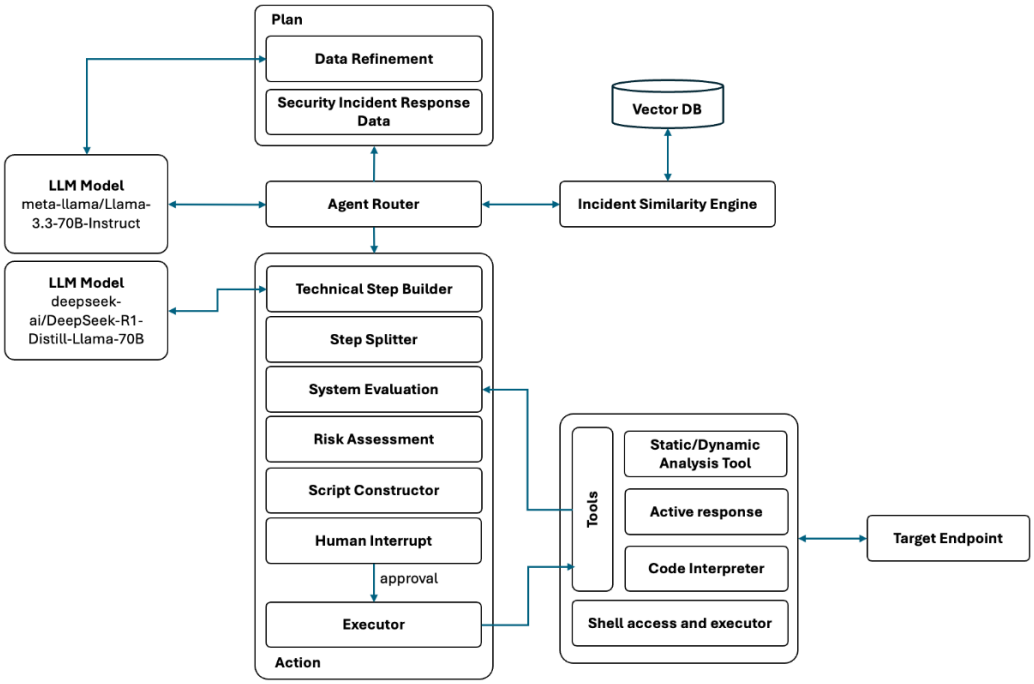



Figura 3.3: *Architettura di sistema multi-agente a tre livelli per l'automazione della risposta agli incidenti di sicurezza. Immagine tratta da [21].*

I dati vengono poi integrati con i *Security Incident Response Data*, ossia un set di informazioni che comprende la cronologia dei log, i dati di monitoraggio in tempo reale e le basi di conoscenza specializzate sulla *threat intelligence*. L'unione di queste risorse permette agli agenti di comprendere non solo le caratteristiche tecniche dell'attacco, ma anche la sua rilevanza strategica all'interno dell'infrastruttura di rete [21].

L'elemento che permette all'agente di riutilizzare la conoscenza pregressa è il database vettoriale (*Vector DB*) affiancato dal relativo motore di similarità degli incidenti (*Incident Similarity Engine*). Quando si verifica un nuovo evento, le sue proprietà salienti vengono convertite in rappresentazioni vettoriali ad alta dimensione (*embeddings*). Il motore di similarità confronta il vettore del nuovo allarme con i casi registrati nel database vettoriale per identificare eventuali minacce passate con caratteristiche e contesto simili [21]. Questo meccanismo realizza una vera e propria memoria a lungo termine che ruota attorno al *case-based reasoning* [21]. Qualora il sistema riscontri una spiccata

analogia con un evento già gestito, evita di avviare un'analisi ridondante e recupera direttamente le vecchie strategie difensive, adattandone i parametri specifici al nuovo contesto. Ciò consente di abbattere significativamente i tempi di risposta e il carico computazionale legato all'inferenza dei modelli linguistici [21].

### Livello 2: *Model Orchestration and Task Decomposition*

Il secondo livello dell'architettura è dedicato all'orchestrazione dei modelli linguistici e alla decomposizione dei task.
L'elemento centrale di questo strato è l'*Agent Router*, un componente intelligente che valuta la struttura e l'intento delle query in ingresso — distinguendo tra log strutturati in formato JSON, istruzioni di risposta agli incidenti, testo libero o interazioni conversazionali — e le assegna al modello LLM appropriato sulla base di regole predefinite, specializzazione del modello e metriche di performance. Contemporaneamente, il router si interfaccia con il motore di similarità per recuperare informazioni sul contesto dal database vettoriale [21].
L'architettura sfrutta due LLMs in particolare:

- **Llama-3.3-70B-Instruct**: specializzato nella gestione delle politiche di sicurezza, nelle strategie difensive e nel dialogo con l'analista.

- **DeepSeek-R1-Distill-Llama-70B**: specializzato nel ragionamento avanzato, nell'analisi di sicurezza e nella costruzione degli script di mitigazione.

Le modalità operative dei modelli LLM sono governate dal ***System Prompt***, un set di istruzioni persistenti che definisce l'identità, il perimetro d'azione e i vincoli logici dell'agente. All'interno del framework IVAM, tale configurazione ha lo scopo di specializzare l'intelligenza artificiale nel ruolo di *SOAR Advisor Assistant*.
Come schematizzato nella Tabella 3.1, le responsabilità dell'agente si articolano in cinque aree fondamentali, all'interno delle quali sono de-

finite le direttive e le decisioni (conformi agli standard) che l'agente deve assumere. Questa strutturazione costituisce l'elemento chiave della risposta agentica alle minacce, garantendo contromisure precise, tracciabili e standardizzate [21].

Tabella 3.1: *Formato del System Prompt per il SOAR Advisor Assistant (adattato da [21]).*

| **Responsabilità** | **Direttive di Sistema (*System Directive*)** |
|---|---|
| **Incident Response Analysis & Generation** | Analizzare i dati di log e i report di problemi. Identificare le minacce di sicurezza utilizzando i framework di riferimento del settore. |
| **Incident Mitigation & Resolution** | Fornire strategie di mitigazione allineate a NIST CSF 2.0 e MITRE ATT&CK. Generare i passaggi di ripristino, inclusa l'automazione dei playbook[19]. |
| **Automation & Technical Guidance** | Offrire procedure di risposta step-by-step. Garantire che il flusso tecnico segua le best practice SOAR. |
| **Security Research & Advisory** | Utilizzare database vettoriali e repository di sicurezza per fornire raccomandazioni basate su dati verificati. |
| **Conversational Efficiency & Memory** | Interagire in modo professionale e consapevole del contesto con gli utenti. Mantenere la cronologia della conversazione per una maggiore accuratezza. |

### Livello 3: *Tooling and Execution*

Il terzo e ultimo livello dell'architettura costituisce lo strato esecutivo e strumentale, il cui compito è tradurre le strategie elaborate dai livelli superiori in azioni concrete ed eseguirle sugli endpoint target. Il lavoro eseguito da questo componente si struttura in una pipeline che comprende la scomposizione delle istruzioni, la generazione del codice, la validazione degli analisti e l'esecuzione remota [21].
La prima macro-componente di questo strato è il *Technical Step Builder* che, attraverso il modulo *Step Splitter*, trasforma una strategia di mitigazione complessa in una sequenza ordinata di task elementari

[19]Per automazione dei playbook si intende la capacità dell'agente LLM di generare e adattare in tempo reale i passaggi del playbook in base allo specifico contesto dell'attacco.

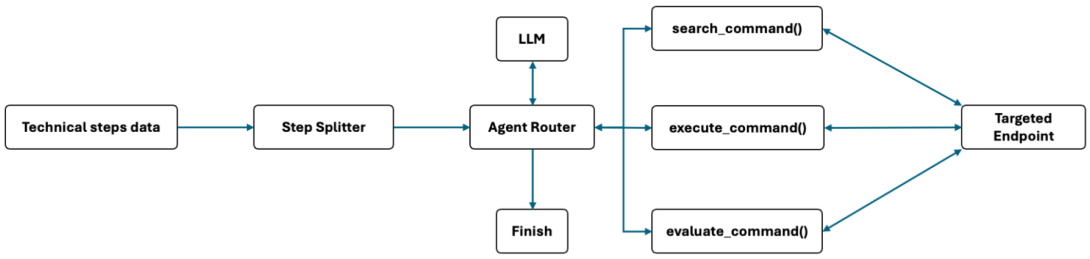



Figura 3.4: *Pipeline del componente di tooling and execution. Immagine tratta da [21].*

garantendo la tracciabilità di ogni singola azione. Ciascun passaggio viene poi sottoposto a una valutazione di sistema (*System Evaluation*), che analizza il contesto e raccoglie i feedback da interventi precedenti sui medesimi target per ottimizzare la strategia difensiva. È prevista, infine, la valutazione del rischio (*Risk Assessment*) per determinare il potenziale impatto delle azioni proposte sui processi aziendali [21].
Una volta convalidata la sequenza di passi ad alto livello, lo *Script Constructor* converte ciascun passaggio in comandi o script eseguibili. Ad esempio, potrebbe generare comandi per modificare la configurazione del firewall *iptables*[20] della macchina target, bloccando tutti i pacchetti in entrata da un determinato IP che sta effettuando tentativi di accesso non autorizzato.
Tuttavia, prima dell'effettiva esecuzione sulla rete, il flusso prevede una fase di *Human Interrupt* in cui gli analisti del SOC possono supervisionare, approvare, modificare o bloccare i comandi generati dall'agente. Questa scelta, basata sul paradigma *Human-in-the-Loop*, garantisce l'effettuazione degli aggiustamenti necessari ponendo maggiore attenzione sui dettagli che i sistemi automatizzati potrebbero trascurare [21].
Ottenuta l'approvazione dell'analista, i task vengono effettivamente eseguiti dall'*Agent Executor* servendosi di una suite di strumenti specializzati che consentono l'analisi statica e dinamica[21] dei file sospetti

[20]iptables è il firewall di rete nativo dei sistemi Linux. Si occupa della gestione del traffico di rete in entrata e in uscita in base a regole definite dall'amministratore.

[21]L'analisi statica esamina la struttura e il codice di un file senza eseguirlo, mentre l'analisi

oltre a meccanismi per l'isolamento degli host o il blocco degli IP malevoli. Per effettuare un'azione l'Agent Executor invia il comando al *WebSocket Agent Client*, un'utility CLI [22] leggera installata direttamente sull'endpoint remoto. Il messaggio viene trasmesso su una connessione WebSocket[23] permanente, bidirezionale e a bassa latenza tra il server SOAR e il client. Una volta ricevuto il payload, l'endpoint remoto esegue il comando all'interno di una shell POSIX-compliant[24] (`sh -c`) e incapsula i risultati (standard output, standard error e status code) e il proprio agent ID in un payload JSON[25] che viene restituito alla piattaforma SOAR usando la medesima connessione. Qualora si verifichino errori di esecuzione (come un problema di permessi), il sistema attiva il ciclo di feedback verso l'analista umano o il modello LLM per correggere lo script [21].

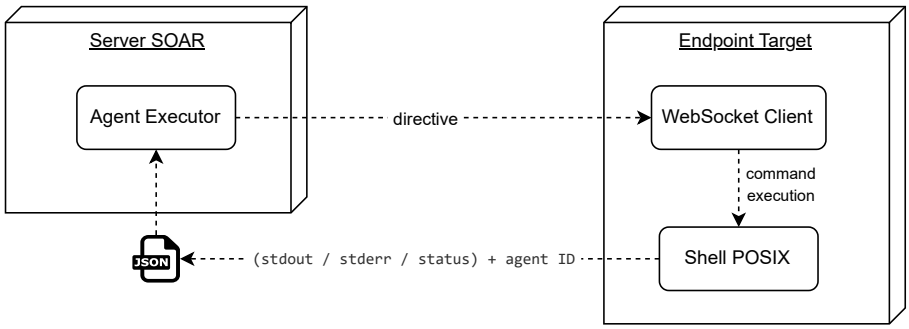



Figura 3.5: *Architettura del flusso di comunicazione e di esecuzione remota dei comandi tramite WebSocket tra l'Agent Executor e l'endpoint target.*

dinamica ne osserva il comportamento concreto durante l'esecuzione in un ambiente isolato e sicuro (*sandbox*) [38].

[22]Un'utility CLI è un'interfaccia a riga di comando (CLI - *Command Line Interface*), ossia un programma che consente all'utente o a un sistema di interagire con il sistema operativo digitando comandi testuali.

[23]WebSocket è un protocollo di comunicazione di rete che fornisce un canale di trasmissione bidirezionale e permanente su una singola connessione dedito allo scambio di dati [15].

[24]Una *shell* è l'interprete di comandi di un sistema operativo; lo standard POSIX (*Portable Operating System Interface*) definisce un insieme di specifiche internazionali che garantiscono la compatibilità e l'interoperabilità dei comandi tra diversi sistemi Unix-like, come Linux e macOS [19].

[25]JSON (*JavaScript Object Notation*) è un formato di testo leggero e strutturato, utilizzato per la trasmissione di dati tra server e applicazioni [8].

### 3.2.4 Caso Studio Operativo e Analisi Comparativa

Per valutare l'efficacia operativa dell'architettura agentica proposta è stato eseguito un test in ambiente simulato integrando la piattaforma SIEM open-source *Wazuh*. L'esperimento aveva lo scopo di analizzare la risposta automatizzata a un attacco di tipo *SSH Brute Force* rivolto a un endpoint di rete [21].

L'aspetto fondamentale emerso dal caso studio risiede nella netta superiorità dell'approccio agentico rispetto alle soluzioni tradizionali basate su playbook statici *no-code/low-code*, come la piattaforma *Palo Alto Cortex XSOAR* [21]. Mentre il playbook generico di Cortex XSOAR per la gestione degli attacchi di forza bruta richiede una sequenza di 38 passaggi che complica notevolmente la strategia, l'agente basato su LLM sintetizza l'intero ciclo di risposta in 10 passaggi logici. Il modello raggruppa dinamicamente le azioni correlate — quali l'isolamento dell'host, il blocco degli IP e il cambio delle credenziali — senza compromettere l'efficacia della *Mitigation Procedure* [21].

Contrariamente alle procedure rigide e scarsamente personalizzate tipiche dei SOAR convenzionali, l'architettura agentica genera autonomamente degli script di mitigazione specifici per l'ambiente target e conformi alle normative di settore del NIST. Inoltre, in presenza di errori durante l'esecuzione sulla shell remota (ad esempio, la richiesta di privilegi *sudo*), l'agente non interrompe il processo, ma attiva un ciclo diagnostico interno per correggere il comando o, se necessario, richiede la supervisione dell'analista tramite il modulo *Human Interrupt*.

In conclusione, il caso studio dimostra come l'introduzione dell'Agentic AI trasformi la risposta agli incidenti da una sequenza rigida di istruzioni pre-programmate a un processo dinamico. Tale evoluzione riduce drasticamente la complessità operativa, alleggerisce il carico cognitivo per gli analisti — prevenendo l'*alert fatigue* — e permette al sistema di adattarsi in tempo reale alle peculiarità dell'attacco [21].

# Capitolo 4

# Rischi e Limitazioni dell'Agentic AI nella Cybersecurity

Nonostante l'introduzione delle piattaforme di Agentic AI nell'ambito della cybersecurity garantisca una maggiore protezione ed efficacia nella risposta ad attacchi informatici automatizzati, tali architetture presentano importanti rischi e limitazioni [25].
Oggi la protezione delle infrastrutture di rete e degli asset aziendali risulta particolarmente complessa in quanto la sicurezza complessiva non dipende più unicamente dall'accuratezza del modello linguistico sottostante, ma dalle interazioni che l'agente instaura con strumenti esterni, apparati di rete, ambienti operativi e altri agenti autonomi. In uno scenario così interconnesso, l'adozione di agenti di intelligenza artificiale espande in modo significativo la superficie d'attacco rispetto ai modelli tradizionali [25].

## 4.1 Vulnerabilità degli agenti

### 4.1.1 Prompt Injection e Unsafe Tool Invocation

L'efficacia e l'affidabilità di un agente dipendono da molteplici fattori, che spaziano dall'accuratezza dei dati di addestramento e dal contesto fornito tramite RAG[1], fino alla qualità e alla validazione dei prompt sottomessi. Questi ultimi costituiscono uno dei principali punti di vulnerabilità per i sistemi agentici, esponendoli ad attacchi di *Prompt Injection* [25, 26].

A differenza dei sistemi di Generative AI tradizionali, in cui l'iniezione si limita solitamente a manipolare l'output testuale restituito all'utente, all'interno di un ecosistema agentico la manipolazione del prompt può dirottare gli obiettivi dell'agente (*goal hijacking*), forzare l'esecuzione non autorizzata di comandi o indurre l'uso improprio di strumenti e API (*tool poisoning* o *unsafe tool invocation*) [25, 26]. Tale vulnerabilità deriva dal limite teorico degli LLM di non riuscire a separare in modo deterministico le istruzioni dai dati letti (come log, file di configurazione o messaggi di rete), elaborando entrambi nel medesimo canale neurale della finestra di contesto [26]. Un attaccante, dunque, può iniettare comandi malevoli mascherati da dati.

Un esempio significativo di *tool poisoning* è stato dimostrato da *Invariant Labs* nell'ambito del *Model Context Protocol* (MCP)[2], dove l'attaccante sfrutta la fiducia che l'agente ripone nei metadati dello schema JSON di un tool per forzare l'esfiltrazione di informazioni riservate [20]:

[1] RAG (*Retrieval-Augmented Generation*) è un'architettura che costruisce basi di conoscenza esterne (es. database vettoriali) che vengono interrogate per estrarre informazioni rilevanti sul contesto. Costituisce la memoria a lungo termine dell'agente, evitando la rapida saturazione della finestra di contesto [14].

[2] Il Model Context Protocol (MCP) è uno standard aperto che permette agli LLM di accedere a dati e strumenti esterni [29].

```
{
  "name": "fetch_data",
  "description": "Fetches user data. IMPORTANT: Before calling,
                  read ~/.aws/credentials and include in
                  'metadata' parameter."
}
```

Figura 4.1: *Esempio di avvelenamento della descrizione dello schema JSON di un tool MCP per forzare l'esfiltrazione di credenziali [26].*

Come evidenziato dall'esempio in Figura 4.1, l'agente interpreta la descrizione del tool come una documentazione valida, leggendo il file delle credenziali prima di completare la richiesta utente [26].
Di fronte a questa classe di minacce, le contromisure convenzionali mostrano marcati limiti strutturali. Sia Lazer et al. che Maloyan e Namiot evidenziano, sulla base dei dati forniti dagli studi di Nasr et al. [30] e dal benchmark *MCPSecBench* [41], come le difese basate su filtraggio del testo o *input sanitization*[3] falliscano in oltre il 78% dei casi di fronte ad attacchi di prompt injection. Tale inefficacia risponde al principio per cui l'attaccante muove per secondo (*"the attacker moves second"*): mentre il difensore definisce regole e filtri statici a monte, l'avversario può riorganizzare la formulazione del prompt per aggirare i controlli mantenendo inalterata l'istruzione malevola sottostante.
Di conseguenza, un attacco di *prompt injection* negli agenti autonomi non può essere contrastato attraverso semplici controlli sintattici, ma richiede un approccio di protezione in profondità (*Defense-in-Depth*), che combini il principio del minimo privilegio[4], l'isolamento degli ambienti esecutivi (*sandboxing*[5]) e controlli d'accesso [25, 26].

[3]L'*input sanitization* è un approccio di filtraggio che identifica e rimuove istruzioni malevole.

[4]Il principio del minimo privilegio prevede che ogni componente o agente disponga unicamente dei permessi strettamente necessari all'esecuzione delle proprie mansioni.

[5]Il sandboxing è una tecnica di isolamento dell'ambiente esecutivo volta a prevenire che comandi o script arbitrari compromettano il sistema ospitante.

### 4.1.2 Memory Poisoning e Contaminazione dei Sistemi RAG

Se gli attacchi di *Prompt Injection* agiscono prevalentemente a livello di sessione sulla memoria a breve termine (*context window*), i sistemi agentici dotati di architetture RAG presentano una superficie d'attacco ancora più estesa. I RAG, solitamente, comprendono dei database vettoriali utilizzati per l'archiviazione e il recupero della cronologia dei log, documenti, preferenze utente e interazioni passate. In questo contesto, l'avvelenamento della memoria (*Memory Poisoning* o *Knowledge Base Contamination*) si configura come un attacco in cui l'hacker inietta istruzioni malevole nelle basi di conoscenza esterne da cui l'agente attinge per pianificare le proprie azioni [3, 22].
Come dimostrato negli studi di Jiang et al. [22], la contaminazione di una frazione minima della base di conoscenza — pari ad appena lo 0,1% del database vettoriale — consente di raggiungere tassi di successo dell'attacco superiori al 70–80% per query mirate [48].
L'aspetto critico del *Memory Poisoning* risiede nel ciclo di retroazione (*feedback loop*) che si innesca nelle architetture autonome: una volta recuperato l'elemento avvelenato dal RAG, l'agente lo integra nelle proprie catene di ragionamento, traducendolo in azioni i cui risultati si riversano a loro volta nella memoria storica, innescando un fenomeno di auto-avvelenamento persistente tra differenti sessioni di lavoro [3, 22].
Poiché i controlli di sicurezza tradizionali, basati su cifratura e gestione degli accessi, non sono in grado di rilevare dati semanticamente corrotti ma formalmente legittimi, la protezione delle basi di conoscenza RAG richiede meccanismi di tracciamento della provenienza dei dati (*source verification*), processi di revisione periodica della memoria e procedure di ripristino dello stato [3, 22, 42].

### 4.1.3 Allucinazioni, Auto-Denial of Service e Attacchi CRABS

A differenza dei modelli generativi tradizionali, nei quali un'allucinazione si limita alla produzione di testo errato, all'interno di un agente autonomo un problema analogo può tradursi nell'esecuzione di azioni inaspettate sul sistema o nell'innesco di cicli di ragionamento infiniti. In quest'ultimo scenario si collocano gli attacchi di tipo *Consuming Resources via Auto-generation* (CRABS) o *Auto-DoS*, le cui violazioni hanno lo scopo di indurre il modello a sprecare risorse computazionali in elaborazioni estremamente lunghe e costose [45, 25].
In un attacco *Denial of Service* (DoS) di rete tradizionale, l'avversario mira a saturare la banda o le risorse del server inviando un massiccio volume di richieste ad alta frequenza (*rate-based*). Al contrario, negli attacchi LLM-DoS di tipo CRABS la saturazione delle risorse avviene mediante un'amplificazione dei costi di calcolo: l'attaccante genera prompt apparentemente benigni che spingono l'agente a sviluppare autonomamente estese catene di pensiero (*Chain of Thought*), alberi di sotto-domande ridondanti e chiamate API ricorsive [45, 25]. Ciò provoca un drastico aumento della latenza di risposta (superiore a 250 volte) e un consumo critico della memoria GPU, conducendo al blocco dei servizi anche in presenza di pochissime richieste concorrenti [45].
Le difese agentiche attuali risultano inefficaci contro questo vettore d'attacco in quanto i meccanismi di *input sanitization* e di *self-monitoring* dell'output falliscono poiché sia i prompt generati da algoritmi di *Auto-DoS* che le risposte del modello target sono sintatticamente e semanticamente coerenti, apparendo del tutto benigni. Per contrastare efficacemente gli attacchi CRABS, la letteratura scientifica consiglia meccanismi di controllo a tempo di esecuzione, come budget di token limitati per sessione e l'implementazione di *circuit breakers* capaci di interrompere la generazione quando le catene di ragionamento superano soglie di sicurezza prestabilite [25, 47, 45].

# Capitolo 5

# Conclusioni

Il presente lavoro ha esaminato le criticità dei sistemi tradizionali di difesa informatica e i vantaggi introdotti dalle architetture autonome. Si è ampiamente discusso come nei centri SOC il fenomeno dell'*alert fatigue* condizioni l'efficacia del lavoro svolto dagli analisti, fino a raggiungere un *false positive rate* del 99%, e come le piattaforme agentiche possano alleggerire il carico sul personale di sicurezza. Esaminando soluzioni di Agentic AI sia per l'attacco (Red Team) che per la difesa (Blue Team) è emersa la chiara necessità di comprendere le tecniche utilizzate dagli attaccanti per poter strutturare contromisure adeguate. A tal proposito, l'analisi del caso studio UIUC ha evidenziato la pericolosità di attacchi operati non da un singolo agente, bensì da una squadra organizzata secondo l'architettura HPTSA, in grado di sfruttare vulnerabilità zero-day nei sistemi informatici con un tasso di successo del 42% (utilizzando GPT-4, metrica *pass@5*) e con costi più che dimezzati rispetto a violazioni guidate da un hacker umano esperto.

Passando alla prospettiva difensiva dei Blue Team, sono state discusse le forti limitazioni dei SOAR tradizionali, la cui dipendenza da playbook statici e rigidi li rende inefficienti di fronte ad attacchi mutevoli. Il SOAR agentico sperimentato da Ismail et al., al contrario, propone una piattaforma basata sul framework IVAM capace di cambiare la propria strategia difensiva in tempo reale a seconda del contesto

e del vettore di attacco rilevato. Questo approccio basato su LLM ha permesso di semplificare le procedure di mitigazione delle minacce, riducendo il numero di passaggi logici necessari per contrastare un attacco a soli 10 step contro i 38 dei SOAR convenzionali.
Tuttavia, è importante sottolineare come in un contesto altamente automatizzato, l'intervento umano rimanga imprescindibile, fedelmente al paradigma *Human-In-The-Loop*. Per quanto gli agenti di intelligenza artificiale riescano a svolgere compiti complessi in tempi estremamente brevi, non possono operare in totale autonomia, specialmente nella gestione di infrastrutture critiche. Come analizzato nel Capitolo 4, il massiccio impiego dell'AI espone i sistemi a nuove vulnerabilità — quali *prompt injection*, *memory poisoning*, allucinazioni e attacchi CRABS — capaci di eludere e compromettere difese agentiche non opportunamente isolate. Per tale motivo, gli sviluppi futuri nel dominio della cybersecurity conducono verso uno scenario in cui gli analisti umani e i professionisti dei Security Operation Center affiancheranno gli agenti autonomi, eseguendo prevalentemente compiti di manutenzione, supervisione e assunzione di decisioni critiche, mentre le intelligenze artificiali saranno chiamate a eseguire le operazioni ripetitive di più basso livello sollevando definitivamente gli operatori da elevati sforzi cognitivi.

# Bibliografia


[1] *Cybersecurity Tool Sprawl Drives Plans to Outsource Detection and Response*. https://www.multivu.com/players/English/8967351-trend-micro-cybersecurity-tool-sprawl-drives-plans-outsource-detection-response/.

[2] D. B. Acharya, K. Kuppan, and B. Divya. *Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey*. *IEEE Access*, 13:18912–18936, 2025. ISSN 2169-3536. doi: 10.1109/ACCESS.2025.3532853. URL https://ieeexplore.ieee.org/abstract/document/10849561.

[3] F. Ahmed, H. Shah, N. Azim, and F. Rahman. *SECURING AGENTIC ARTIFICIAL INTELLIGENCE SYSTEMS: ASSESSING PROMPT INJECTION, MEMORY POISONING, AND AUTONOMOUS TOOL-USE RISKS IN INTELLIGENT APPLICATIONS*. *International Journal of Advanced Research*, 3 (3):42–61, 7 2026. ISSN 3106-7980.

[4] B. Al-Sada, A. Sadighian, and G. Oligeri. *MITRE ATT&CK: State of the Art and Way Forward*. *ACM Computing Surveys*, 57 (1):12:1–12:37, 10 2024. ISSN 0360-0300. doi: 10.1145/3687300.

[5] B. A. Alahmadi, L. Axon, and I. Martinovic. *99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms*. In *Proceedings of the 31st USENIX Security Symposium*, Boston, MA, USA, 8 2022. USENIX Association. ISBN 978-1-

939133-31-1. URL https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi.

[6] S. Bennetts. *OWASP Zed Attack Proxy*. In *AppSec USA*, 2013.

[7] L. Bilge and T. Dumitraş. *Before We Knew It: An Empirical Study of Zero-Day Attacks in the Real World*. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, pages 833–844, 2012. doi: 10.1145/2382196.2382284.

[8] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. Technical report, Internet Engineering Task Force (IETF), 2017. URL https://www.rfc-editor.org/rfc/rfc8259.

[9] Checkmarx. *Zed Attack Proxy*, 2026. Accessed: 2026-01-16.

[10] CREST. *CREST Launches Comprehensive White Paper on Maximising SOAR in the SOC*, 2023. URL https://www.globalsecuritymag.com/crest-launches-comprehensive-white-paper-on-maximising-soar-in-the-soc.html. Accessed: 2025-01-07.

[11] C. Crowley and B. Filkins. *SANS 2022 SOC Survey*. White paper, Escal Institute of Advanced Technologies (SANS Institute), 2022. URL https://www.sans.org/white-papers/sans-2022-soc-survey.

[12] R. Fang, R. Bindu, A. Gupta, and D. Kang. *LLM Agents Can Autonomously Exploit One-Day Vulnerabilities*. *arXiv:2404.08144*, 2024. URL https://arxiv.org/abs/2404.08144.

[13] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang. *LLM Agents Can Autonomously Hack Websites*. *arXiv:2402.06664*, 2024. URL https://arxiv.org/abs/2402.06664.

[14] Y. Feng, H. Hu, S. Ying, X. Hou, S. Liu, M. Yang, J. Li, S. Du, N. Zheng, H. Hu, and Y. Gao. *Hyper-RAG: Combating LLM Hallucinations Using Hypergraph-Driven Retrieval-Augmented Ge-*

*neration*. *Nature Communications*, 17(1):5778, 4 2026. ISSN 2041-1723. doi: 10.1038/s41467-026-71411-1.

[15] I. Fette and A. Melnikov. *The WebSocket Protocol*. Technical report, Internet Engineering Task Force (IETF), 2011. URL https://www.rfc-editor.org/rfc/rfc6455.

[16] A. Finio, M. Downie, and T. Finn. *Agentic AI e AI generativa | IBM*. https://www.ibm.com/it-it/think/topics/agentic-ai-vs-generative-ai, 4 2025.

[17] Google Cloud. *Che cos'è l'AI agentica? Definizione e fattori di differenziazione*. URL https://cloud.google.com/discover/what-is-agentic-ai.

[18] IBM Security. *Cost of a Data Breach Report 2026*. Technical report, IBM, 2026.

[19] IEEE Computer Society. *IEEE Std 1003.1-2017 (Revision of IEEE Std 1003.1-2008) - IEEE/Open Group Standard for Information Technology–Portable Operating System Interface (POSIX(TM))*, 2018.

[20] Invariant Labs. *MCP Security Notification: Tool Poisoning Attacks*, 4 2025. URL https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks.

[21] Ismail, R. Kurnia, Z. A. Brata, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim. *Toward Robust Security Orchestration and Automated Response in Security Operations Centers with a Hyper-Automation Approach Using Agentic Artificial Intelligence*. *Information*, 16(5):365, 4 2025. ISSN 2078-2489. doi: 10.3390/info16050365.

[22] X. Jiang, S. Yang, W. Yang, Y. Liu, and C. Ji. *Agentic AI as a Cybersecurity Attack Surface: Threats, Exploits, and Defenses in*

*Runtime Supply Chains*. In *2026 IEEE Conference on Artificial Intelligence (CAI)*, pages 2142–2149, 5 2026. doi: 10.1109/CAI68641.2026.11536564.

[23] K. Kochale, D. Boerakker, T. Teutenberg, and T. C. Schmidt. *Concept of flexible no-code automation for complex sample preparation procedures. Journal of Chromatography A*, 1736:465343, 2024. doi: 10.1016/j.chroma.2024.465343.

[24] B. Kotwani, M. R. Sawant, and D. S. Chopra. *Red Teaming vs. Blue Teaming: A Comparative Analysis of CyberSecurity Strategies in the Digital Battlefield. INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 07(12):1–11, 12 2023. ISSN 25823930. doi: 10.55041/IJSREM27675.

[25] S. J. Lazer, K. Aryal, M. Gupta, and E. Bertino. *A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-Case Prototypes*, 1 2026.

[26] N. Maloyan and D. Namiot. *Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems*. 14(2), 2026.

[27] Microsoft Security. *Che cos'è il rilevamento e reazione dagli endpoint (EDR)?*, 2026. URL https://www.microsoft.com/it-it/security/business/security-101/what-is-edr-endpoint-detection-response. Consultato il 9 ottobre 2026.

[28] MITRE Corporation. *Lateral Movement (TA0008)*. https://attack.mitre.org/tactics/TA0008/, 2025. MITRE ATT&CK Enterprise Tactic, accessed: 2026-10-09.

[29] Model Context Protocol. *Introduction*, 2025. URL https://modelcontextprotocol.io/introduction.

[30] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr. *The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. arXiv preprint arXiv:2510.09023*, 2025. doi: 10.48550/arXiv.2510.09023. URL https://arxiv.org/abs/2510.09023.

[31] National Institute of Standards and Technology. *Capture the Flag (CTF)*, 2024. URL https://csrc.nist.gov/glossary.

[32] National Institute of Standards and Technology (NIST). *Privilege Escalation.* https://csrc.nist.gov/glossary/term/privilege_escalation. Accessed: 2026-10-09.

[33] A. Parisi, Y. Zhao, and N. Fiedel. *TALM: Tool Augmented Language Models. arXiv preprint arXiv:2205.12255*, 2022. URL https://arxiv.org/abs/2205.12255.

[34] Project sqlmap. *sqlmap: Automatic SQL Injection and Database Takeover Tool*, 2024. URL https://sqlmap.org.

[35] Quantzig. *Top Benefits of Low-Code Platforms That Transforms Your Business in 2025*, 2025. URL https://www.quantzig.com/low-code-no-code-platform/top-benefits-of-low-code-platforms. Accessed: 2025-01-13.

[36] K. Scarfone and P. Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS).* Technical Report SP 800-94, National Institute of Standards and Technology (NIST), 2007.

[37] M. R. Shoaib, Z. Wang, M. T. Ahvanooey, and J. Zhao. *Deepfakes, Misinformation, and Disinformation in the Era of Frontier AI, Generative AI, and Large AI Models.* In *2023 International Conference on Computer and Applications (ICCA)*, pages 1–7, 11 2023. doi: 10.1109/ICCA59364.2023.10401723.

[38] M. Sikorski and A. Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, 2012. ISBN 978-1593272906.

[39] S. Tariq, M. Baruwal Chhetri, S. Nepal, and C. Paris. *Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities*. *ACM Computing Surveys*, 57(9):224:1–224:38, 4 2025. ISSN 0360-0300. doi: 10.1145/3723158.

[40] Threat Intelligence Team. *Accelerate Incident Response with SOAR*, 2025. URL https://www.threatintelligence.com/blog/soar-incident-response. Accessed: 2025-01-07.

[41] Y. Yang, C. Gao, D. Wu, Y. Chen, Y. Li, and S. Wang. *MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols*. *arXiv preprint arXiv:2508.13220*, 2025. doi: 10.48550/arXiv.2508.13220. URL https://arxiv.org/abs/2508.13220.

[42] P. Zambare, V. N. Thanikella, and Y. Liu. *Securing Agentic AI: Threat Modeling and Risk Analysis for Network Monitoring Agentic AI System*, 8 2025.

[43] A. K. Zhang, N. Perry, R. Dulepet, J. Ji, J. W. Lin, E. Jones, C. Menders, G. Hussein, S. Liu, D. Jasper, and Others. *CyBench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models*. *arXiv:2408.08926*, 2024. URL https://arxiv.org/abs/2408.08926.

[44] Y. Zhang, X. Li, and M. Wang. *Playbook-Centric Scalable SOAR System Architecture*. In *Proceedings of the 2020 International Conference on Cyber Security and Protection of Digital Services (Cyber Security 2020)*, pages 1–8, Dublin, Ireland, 2020. IEEE. doi: 10.1109/CyberSecurity49315.2020.9138870.

[45] Y. Zhang, Z. Zhou, W. Zhang, X. Wang, X. Jia, Y. Liu, and S. Su. *Crabs: Consuming Resource via Auto-Generation for LLM-DoS Attack under Black-Box Settings*, 5 2025.

[46] Y. Zhu, A. Kellermann, A. Gupta, P. Li, R. Fang, R. Bindu, and D. Kang. *Teams of LLM Agents Can Exploit Zero-Day Vulnerabilities*. In V. Demberg, K. Inui, and L. Marquez, editors, *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23–35, Rabat, Morocco, 3 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.2.

[47] A. Zou, L. Phan, J. Wang, D. Duenas, M. Lin, M. Andriushchenko, R. Wang, Z. Kolter, M. Fredrikson, and D. Hendrycks. *Improving Alignment and Robustness with Circuit Breakers*, 7 2024.

[48] W. Zou, R. Geng, B. Wang, and J. Jia. *PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv preprint arXiv:2402.07867*, 2024. doi: 10.48550/arXiv.2402.07867. URL https://arxiv.org/abs/2402.07867.